전자전기컴퓨터설계실험II

STM32_2 결과보고서

PC·MCU·FPGA UART 통신
서보모터 · SRAM 읽기·쓰기 · DAC

실험 정보
실험일: 2026년 10월 26일
이름: 주윤찬
학번: 2023440132
실험 장비: NUCLEO-G474RE / Combo II-DLD

제출 자료
https://github.com/YunchanJu/STM32_2
소스·시연 자료 기준 태그:
stm32-2-results-20261026-v1
기준 커밋(40자리):
fea589ee901852c92caa34989f6e87efc100fd87
보고서에는 과제5·6·7의 통신 화면과 FPGA 실험1·2·3의 시뮬레이션 파형을 분석하고, 네 개의 보드 영상에서 추출한 사진을 함께 수록하였다.
디지털 시뮬레이션과 실제 보드에서 관찰한 결과를 구분하여 기술하였다.

1. 설계 목표와 실험 방법

1.1 실험 목표

10/26 실습은 PC의 입력 문자를 MCU에서 확인하고, 같은 바이트를 FPGA까지 전달한 뒤 LED 명령으로 해석하는 과정이다. 과제5, 과제6, 과제7의 순서로 데이터 경로를 확장하였다.[1]

FPGA 개별 실습은 서보 펄스 생성, 외부 SRAM의 읽기·쓰기, 외부 DAC의 디지털 제어를 대상으로 한다. 각 실습에서 논리 파형의 예상 결과와 실제 보드 영상을 비교한다. STM32 내부 ADC·DAC를 사용하는 실험은 아니다.[1,2]

1.2 장치와 연결

구간	연결 및 설정
PC ↔ MCU	ST-LINK VCP / UART2
MCU → FPGA	PC10(TX) → 확장 GPIO RX
FPGA → PC	F6 → 보드 USB-UART
공통 조건	GND 공통 / 3.3 V 신호
통신 형식	9600 baud / 8N1

UART3 PC11(RX)은 MCU가 FPGA의 응답을 받을 때 필요한 추가 연결이다. 이번 일방향 전달에서 FPGA의 수신값은 별도 USB-UART 포트로 확인한다.[1]

1.3 UART의 기본 구조

UART는 데이터와 함께 클록을 보내지 않으므로 송수신기가 같은 전송률과 프레임 형식을 사용해야 한다. 수신, 송신, 전체 UART의 구성은 비동기 직렬 인터페이스 설계의 기본 구조와 연결된다.[3, 제8장]

8N1 한 프레임은 시작 1비트, 데이터 8비트, 정지 1비트로 총 10비트이다. 9600 baud에서 한 비트는 약 104.17 μs, 한 바이트 프레임은 약 1.042 ms이다.

1.4 구현과 확인 순서

MCU는 getchar()의 반환값을 int로 받아 EOF를 먼저 검사한다. 정상 입력만 uint8_t로 변환하여 처리한다. 과제5는 문자·16진수를 출력하고, 과제6·7은 UART3로 한 바이트씩 전달한다. UART2 폴링 입력과 UART2 인터럽트 수신을 동시에 사용하지 않는다.[1]

```
int ch = getchar();
if (ch == EOF) return;
uint8_t data = (uint8_t)ch;
HAL_StatusTypeDef result =
  HAL_UART_Transmit(
    &huart3, &data, 1U, 100U);
```

HAL_OK(0)는 MCU 송신 함수의 정상 종료를 뜻한다. FPGA의 실제 수신은 FPGA COM 포트의 문자 출력과 별도로 대조한다. 결과 0만으로 전체 통신 경로의 성공을 판정하지 않는다.[1]

1.5 FPGA 핀과 재현 조건

FPGA의 기존 USB-UART RX 핀 C6는 MCU 점퍼선 입력이 아니다. 확장 GPIO의 실제 핀을 장비 핀맵에서 확인하여 XDC의 RX PACKAGE_PIN으로 지정해야 한다. PC 출력용 TX F6는 유지할 수 있다.[1]

제출 소스의 clock_only XDC에는 장치별 실제 배선 핀 전체가 정의되어 있지 않다. 따라서 보드 재현에는 실제 연결한 확장 RX, LED 및 외부 장치 핀 제약의 확인이 필요하다. 사진만으로 확인되지 않는 패키지 핀 번호는 기재하지 않았다.

1.6 자료와 판정 기준

시뮬레이션은 파형·로그의 수치로, 보드 동작은 영상의 관찰 내용으로 판정한다. 영상에서 측정되지 않은 각도, 펄스 폭, SRAM 접근 시간, DAC 출력 전압은 실측값으로 취급하지 않는다.

제출 소스는 이 보고서의 기준 버전이다. 촬영 당시 실행 바이너리와 제출 소스가 완전히 동일하다는 것을 확인할 별도의 기록은 없으므로, 소스 검토·빌드 결과와 촬영 결과를 구분한다.

2. 과제5·6 - 문자 수신과 FPGA 전달

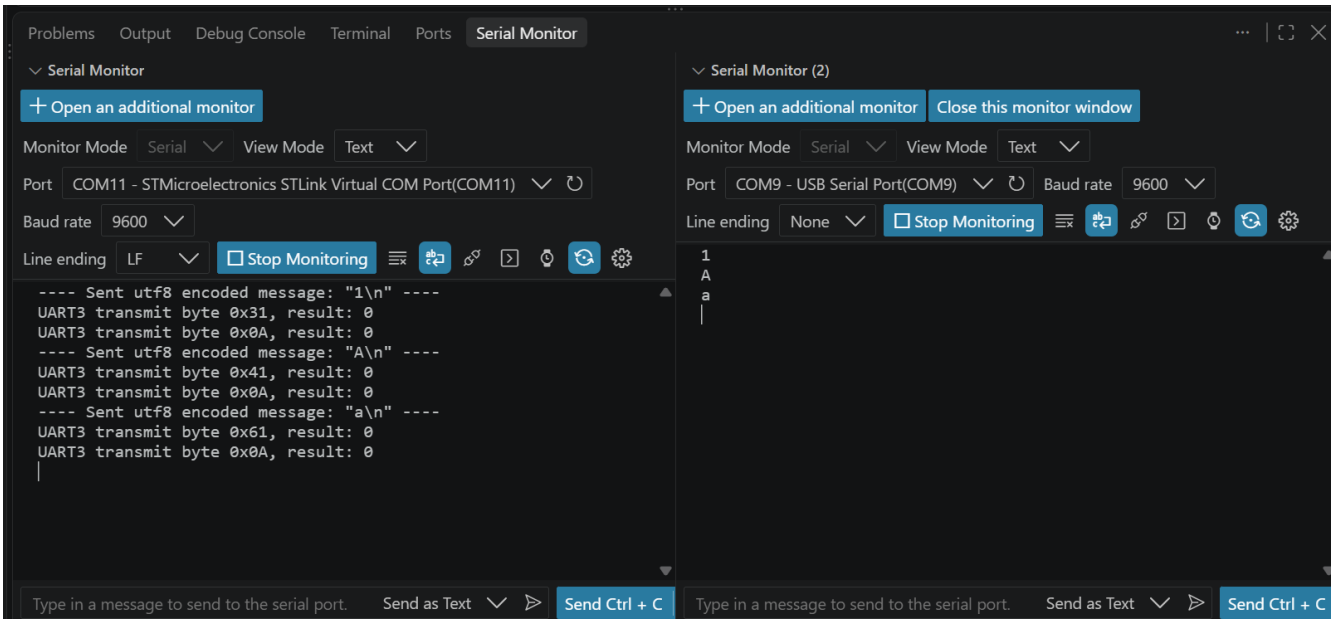


그림 1. 과제6의 두 Serial Monitor. 왼쪽 MCU COM11의 전송 로그와 오른쪽 FPGA COM9의 수신 문자.

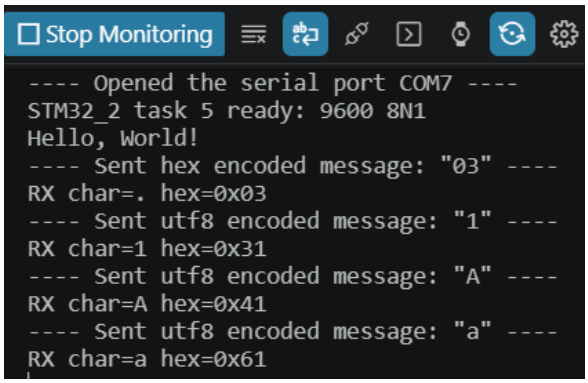
2.1 과제5: 문자와 바이트 값

PC의 Serial Monitor에서 입력한 값을 UART2로 수신하고 문자와 16진수 바이트를 출력하였다. 한 문자 시험의 교안 설정은 줄바꿈 없음(None)이다.[1]

입력 방식·값	수신 값	판정
문자 1	0x31	일치
문자 A	0x41	일치
문자 a	0x61	일치
Hex 03	0x03 / .	제어 바이트

세 문자는 교안의 예상값과 일치하였다. Hex 입력 03은 숫자 문자 3(0x33)이 아니라 제어 바이트 0x03이다. 화면의 점(.)은 출력 불가능한 문자를 표시한 것으로, 원래 수신값을 점의 ASCII 값으로 해석하면 안 된다.

입력이 없을 때 getchar()에서 대기하는 상태는 정상이다. 중단 화면만으로 프로그램 오류라고 판단하지 않고, 문자 입력 후 로그가 추가되는지 확인하는 것이 적절하다.



2.2 과제6: 송신과 수신의 대조

MCU VCP는 COM11, FPGA USB-UART는 COM9로 열려 있으며 두 포트 모두 9600 baud이다. MCU가 1, A, a를 보낸 뒤 FPGA 모니터에도 같은 문자가 각각 출력되었다. MCU 송신 로그의 결과값은 0이다.

실제 화면의 MCU 보내기 설정은 LF이다. 따라서 문자 바이트 0x31·0x41·0x61 뒤에 0x0A도 전송되었다. FPGA 모니터의 줄이 바뀌는 동작은 이 줄바꿈과 함께 해석해야 한다. 단일 문자 검증을 반복할 때는 교안대로 None을 선택하여 추가 바이트를 배제한다.[1]

2.3 핵심 처리

```
#if STM32_2_STAGE == 5
printf("RX char=%c hex=0x%02X\r\n",
       printable, (unsigned)data);
#else
status = HAL_UART_Transmit(
    &huart3, &data, 1U, 100U);
#endif
```

위 코드는 처리 흐름의 발췌이다. 전체 구현은 ece2_474_routine.c에 있다. CMake의 Task5·Task6·Task7 설정으로 기능을 선택하며 이전 주차의 LED 타이머 동작을 입력 처리와 혼합하지 않는다.

3. 과제7 - UART 명령에 따른 LED 제어

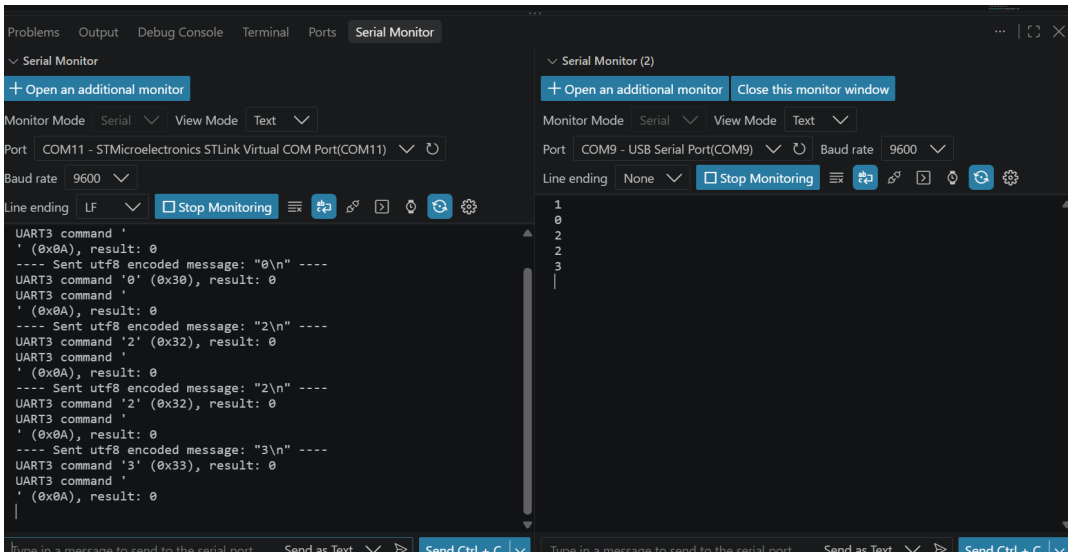
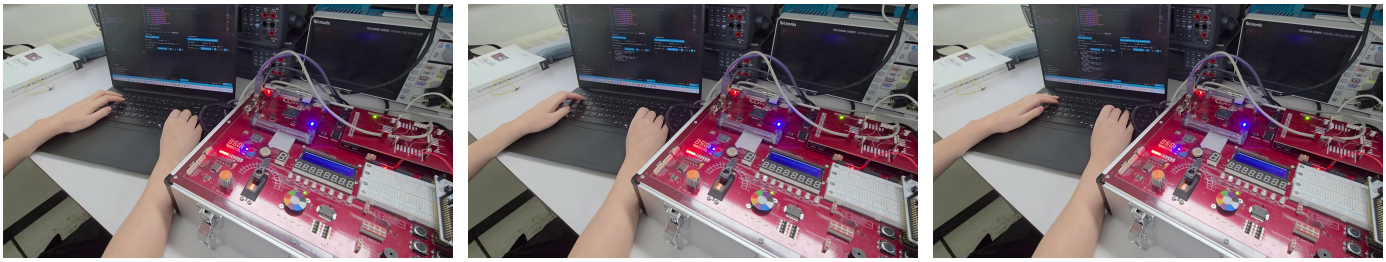


그림 2. 과제7의 명령 전달 화면. COM11의 1, 0, 2, 2, 3 입력과 COM9 수신 출력.



0.17 s 3.28 s 7.51 s

그림 3. 과제7 보드 영상의 일부. 명령 입력 환경과 FPGA LED 표시를 확인할 수 있다.

3.1 명령 해석과 예상 상태

문자	코드	동작
0	0x30	LED 끄기
1	0x31	LED 켜기
2	0x32	LED 반전
그 외	예: 0x33, 0x0A	기존 상태 유지

화면 입력 순서 1 → 0 → 2 → 2 → 3의 예상 LED 상태는 켜짐 → 꺼짐 → 켜짐 → 꺼짐 → 꺼짐이다. 1은 설정 명령이므로 반복해도 켜짐을 유지하고, 2는 반전 명령이므로 매번 상태가 바뀌어야 한다.[1]

LF(0x0A)와 숫자 바이트 0x01은 문자 1(0x31)과 다르다. 유효 명령 외에는 상태를 유지하여 줄바꿈이 추가되어도 의도하지 않은 LED 변화가 생기지 않게 한다.

3.2 회로 구조와 관찰 결과

```
if (rx_valid) begin
  case (rx_data)
    8'h30: led <= 1'b0;
    8'h31: led <= 1'b1;
    8'h32: led <= ~led;
    default: led <= led;
  endcase
end
```

MCU는 입력 바이트를 전달하고 FPGA가 명령을 해석한다. FPGA UART는 수신 데이터를 PC로 에코하여 입력 경로를 확인한다. 50 MHz에서 비트당 5208클록을 사용하면 약 9600.61 baud로, 목표값 대비 차이는 약 0.0064%이다.

수신 화면에는 1, 0, 2, 2, 3이 순서대로 나타났고, 보드 영상에서는 LED 표시가 변하는 모습을 확인하였다. 영상의 문자와 특정 LED를 프레임마다 대응하기 어려워 위 예상 상태열 전체를 실측 상태열로 단정하지 않았다.

전체 회로·시험 입력: uart_led.v / tb_uart_led.sv.

4. FPGA 실험1 - 서보모터 제어

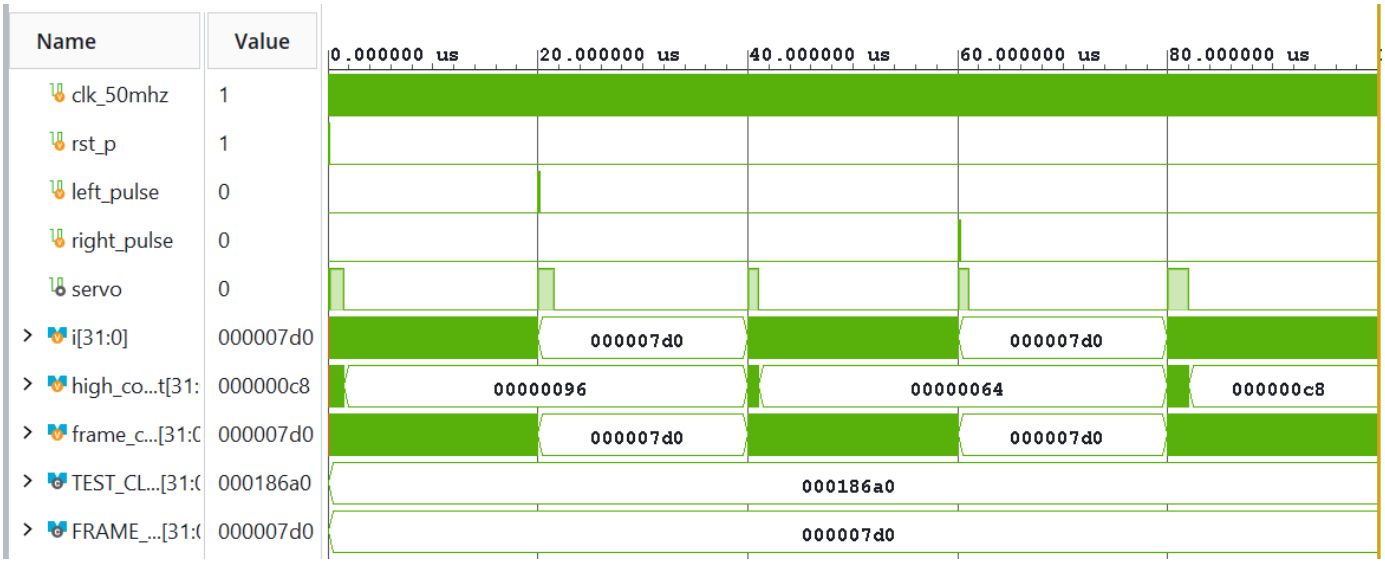
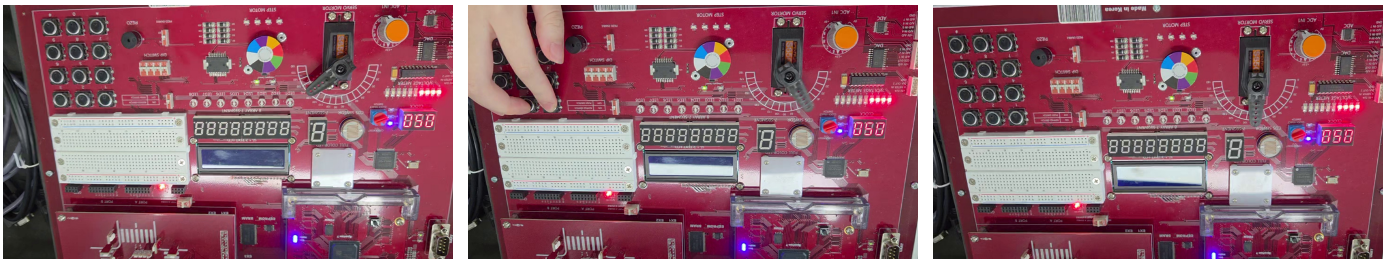


그림 4. 서보 시뮬레이션. 프레임 카운트 0x7D0(2000), HIGH 카운트 0x96·0x64·0xC8(150·100·200).

```
LAB6_SERVO_PASS frame=2000 widths=100/150/200 limits=1000..2000
$finish called at time : 100036 ns : File "C:/Users/yunji/ece2/ece2_2026/LAB6/FPGA/01_servo/sim/tb_servo.sv" Line 86
```

그림 5. 서보 시험 로그: frame=2000, widths=100/150/200, limits=1000..2000.



0.16 s 3.07 s 7.02 s

그림 6. 서보 보드 영상에서 추출한 세 시점. 버튼 조작 전후로 모터 축의 방향이 달라진다.

4.1 설계와 예상 파형

일정한 프레임 안에서 HIGH 펄스 폭을 바꾸어 서보 위치를 제어한다. 초기 폭은 1500 µs이며 입력 펄스에 따라 100 µs씩 변화하고, 1000~2000 µs로 제한한다. 동시 좌·우 입력에서는 유지한다.[2]

프레임 중간에서 폭이 바뀌어 불완전한 펄스가 생기지 않도록 새 폭을 프레임 경계에서 적용한다. 50 MHz 기준 20 ms는 1,000,000클록, 1·1.5·2 ms는 각각 50,000·75,000·100,000클록이다.

4.2 축소 시뮬레이션 시간

시험의 CLK_HZ 매개변수는 100,000이고 프레임 길이는 2000클록이다. TB의 실제 클록 주기는 10 ns이므로 파형의 프레임 간격은 20 µs, HIGH 폭은 1·1.5·2 µs이다. 파형의 µs를 보드의 ms 실측값으로 바꾸어 해석하면 안 된다.

4.3 결과 비교

로그는 2000클록 프레임, 100·150·200클록 펄스 폭과 제한값 시험의 PASS를 표시한다. 파형에서도 HIGH 카운트가 150에서 100 또는 200으로 변하는 구간을 볼 수 있다. 초기 폭과 입력 변화는 설계한 논리 동작에 부합한다.

실제 영상에서는 제어 버튼을 조작하는 동안 서보 축의 방향이 달라졌다. 이는 펄스 폭에 따른 위치 제어의 정성적 결과와 부합한다. 각도 눈금이나 펄스 폭 측정 화면이 없으므로 회전 각도와 실제 20 ms 주기는 수치 결과로 제시하지 않는다.

4.4 결과의 범위

시뮬레이션은 펄스의 클록 수와 상·하한을 확인하며, 실제 모터의 부하·전원·기구 오차까지 검증하지는 않는다. 보드에서 각도를 정량 비교하려면 각도 기준과 오실로스코프 펄스 측정을 함께 기록해야 한다.

소스: servo_ctrl.v

시험: tb_servo_ctrl.sv

5. FPGA 실험2 - SRAM 읽기·쓰기

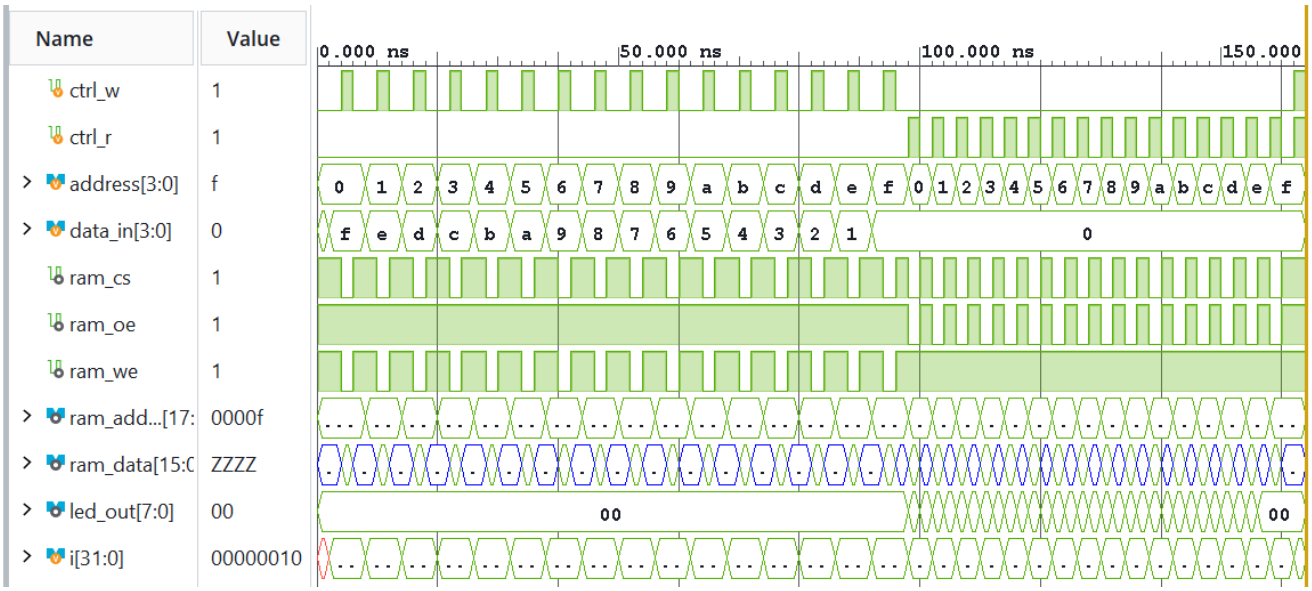
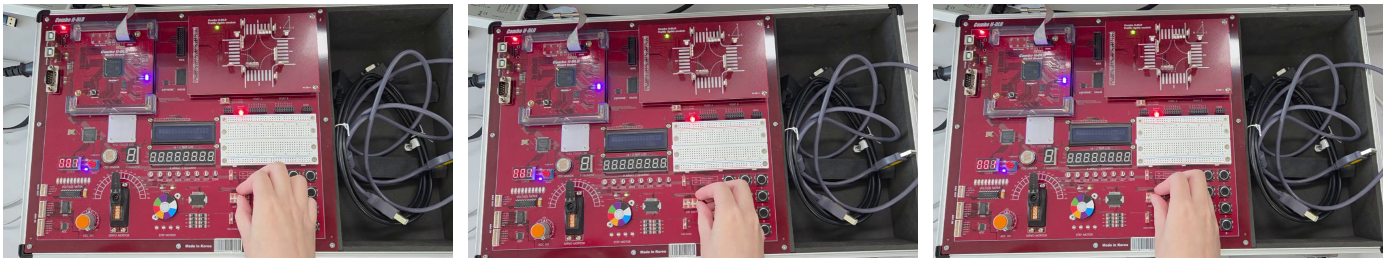


그림 7. SRAM 전체 파형. 주소 0~F에 F~0을 기록한 후 같은 주소들을 읽는다.

```
The current simulation has finished. Please use 'restart' to begin a new simulation.
$finish called at time : 164 ns : File "C:/Users/yunji/ece2/ece2_2026/LAB6/FPGA/02_sram/sim/tb_sram.sv" Line 86
```

그림 8. SRAM 시뮬레이션은 164 ns에서 종료되었다. 이 종료 메시지 자체는 PASS 표시가 아니다.



4.07 s 16.95 s 29.49 s

그림 9. SRAM 보드 영상 일부. 스위치·버튼 조작과 LED 표시를 관찰하였다.

5.1 버스와 제어 신호

SRAM은 주소에 대응하는 데이터를 쓰고 같은 주소에서 읽는 방식으로 확인한다. 주소·데이터·제어 신호를 구분하는 구성은 외부 SRAM 제어의 기본 구조이다.[3, 제11장]

동작	CS/OE/WE	데이터 구동
쓰기	0 / 1 / 0	FPGA → SRAM
읽기	0 / 0 / 1	SRAM → FPGA
비선택	1 / 1 / 1	고임피던스(Z)

쓰기에만 FPGA가 양방향 데이터 버스를 구동하고, 읽기·비선택 상태에서는 Z로 해제한다. 읽기와 쓰기를 동시에 요청하면 접근을 수행하지 않도록 한다. 주소 상위 비트는 0으로 두며 시험 데이터는 하위 4비트를 사용한다.[2]

쓰기 중 FPGA와 SRAM이 동시에 버스를 구동하면 충돌이 생긴다. 따라서 OE·WE의 활성 조건과 버스 소유권을 함께 확인해야 한다.

5.2 파형 분석과 실제 결과

쓰기 구간의 주소는 0, 1, ..., F로 증가하고 입력 데이터는 F, E, ..., 0으로 감소한다. 읽기 구간에서도 같은 주소 순서를 사용한다. 확대 파형의 led_out에는 0F, 0E, 0D, 0C, 0B 등이 나타난다. 접근 사이의 Z는 버스 해제를 뜻한다.

종료 로그에는 164 ns에서 $finish가 실행되었다는 정보만 있다. 따라서 로그만으로 16개 주소의 자동 비교가 모두 통과했다고 볼 수 없으며, 저장·읽기 일치는 파형에서 확인 가능한 구간을 근거로 판단하였다.

보드 영상은 입력 조작과 LED 표시 변화를 보여 준다. 다만 영상만으로 모든 주소와 LED 값을 명확히 판독할 수 없어 실제 SRAM 전체 주소의 수치 대조표는 작성하지 않았다.

5.3 타이밍 검증의 한계

디지털 SRAM 모델은 실제 부품의 접근 시간, 데이터 유지 시간, 배선 지연을 완전히 재현하지 않는다. 보드에서 안정적인 읽기를 확인하려면 장치 데이터시트의 시간 조건과 실제 파형을 추가로 비교해야 한다.

소스: sram.v / tb_sram.sv

6. FPGA 실험3 - 외부 DAC 제어

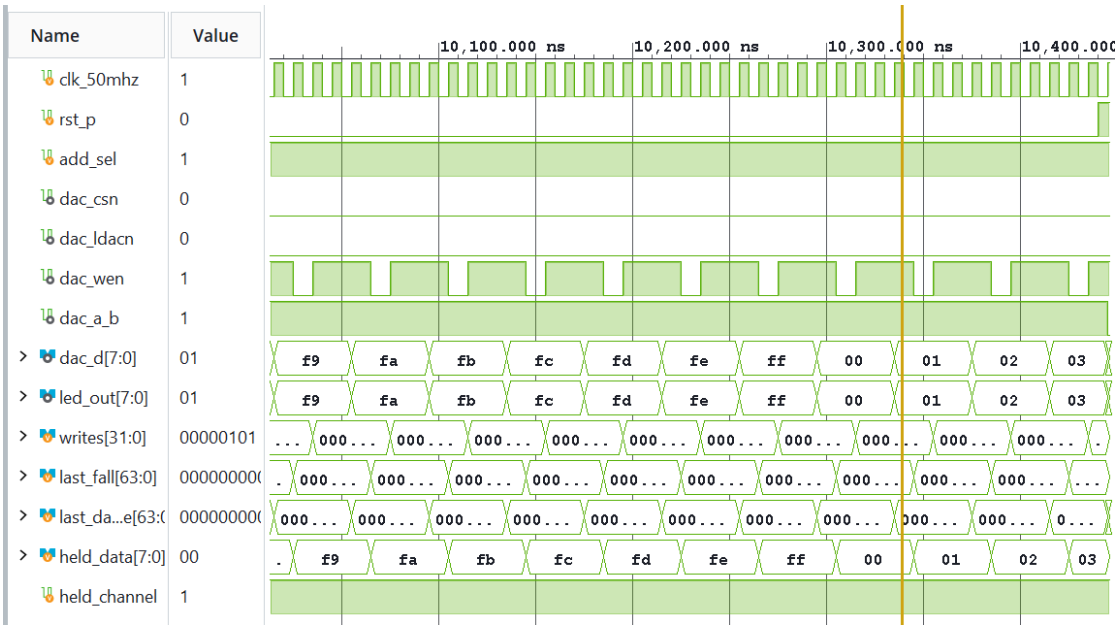
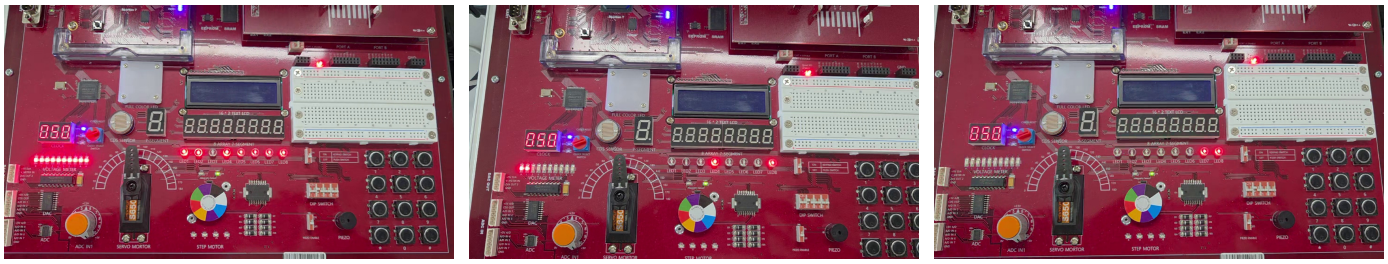


그림 10. DAC 상세 파형. 디지털 입력과 LED 출력이 F9~FF, 00~03으로 변하며 write 펄스가 반복된다.

```
LAB6_DAC_PASS writes=260 wrap=255_to_0 setup_hold channel
$finish called at time : 10446 ns : File "C:/Users/yunji/ece2/ece2_2026/LAB6/FPGA/03_dac/sim/tb_dac.sv" Line 67
```

그림 11. DAC 시험 로그: 260회 쓰기, 255 → 0 순환, setup/hold 및 채널 시험 PASS.



5.76 s 24.00 s 41.76 s

그림 12. DAC 보드 영상의 세 시점. 입력 조작에 따른 보드의 디지털 표시를 확인하였다.

6.1 설계 목표와 제어

8비트 디지털 값을 증가시키며 외부 DAC에 기록하고, 채널 선택과 쓰기 신호의 관계를 확인한다. CS와 LDAC은 LOW로 유지하고 WE는 한 클록 LOW 펄스로 사용한다. 쓰기 구간에는 데이터와 채널을 안정적으로 유지한다.[2]

기본 설계의 50 MHz 및 UPDATE_CYCLES=500,000 조건에서는 10 ms마다 값이 갱신된다. 256단계 순환에는 2.56 s가 필요하며 반복 주파수는 약 0.3906 Hz이다. 이는 코드 매개변수에 따른 예상값이다.

6.2 시뮬레이션 분석

TB는 UPDATE_CYCLES=4와 10 ns 클록을 사용하므로 갱신 간격은 40 ns이다. 260회의 쓰기와 종료 시각 10,446 ns는 이 축소 시험 조건으로 해석한다. 실제 보드의 10 ms 갱신을 측정한 결과가 아니다.

6.3 결과와 교안 목표 비교

상세 파형에서 dac_d와 led_out은 F9, FA, ..., FF, 00, 01, 02, 03 순서로 일치한다. 255 다음에 0으로 순환하는 8비트 카운터 동작을 확인할 수 있다. 전체 파형에는 A/B 채널 선택의 변화가 보이며 로그도 데이터·채널 유지 시험의 통과를 표시한다.

보드 영상에서는 입력 조작 중 디지털 LED 패턴이 달라졌다. 시뮬레이션의 디지털 입력·쓰기 신호와 보드의 표시 변화를 구분하여 비교하였다. 이 실험의 결과 분석은 사진·영상에서 관찰한 디지털 제어 동작을 대상으로 한다.

소스: dac.v / tb_dac.sv

7. 종합 고찰 및 결론

7.1 예상과 실제의 비교

항목	확인한 결과
과제5	1/A/a의 0x31/41/61 일치
과제6	별도 COM에서 1/A/a 수신
과제7	명령 수신·보드 LED 변화
서보	축소 펄스 시험·축 이동
SRAM	쓰기/읽기 파형·보드 표시
DAC	디지털 순환·채널 제어

PC → MCU → FPGA로 경로를 늘릴 때 각 장치의 출력을 나누어 확인하는 방식이 유효하였다. MCU 함수의 반환값, FPGA의 에코 문자, 보드 LED는 서로 다른 검증 지점을 나타낸다. 입력이 수신되었다는 사실과 최종 장치의 물리량이 정확하다는 사실은 구분해야 한다.

7.2 문제 분석과 개선

첫째, 문자와 수치가 혼동될 수 있다. 과제5의 16진수 출력으로 0x03과 문자 3, 0x01과 문자 1을 구분하였다. 둘째, LF 설정으로 0x0A가 추가되므로 단일 문자 시험에서는 None을 선택하고, 명령 해석기는 다른 문자를 무시해야 한다.

셋째, MCU의 송신 성공만으로 FPGA 수신을 추정하지 않고 별도 COM 출력을 비교하였다. 연결 확인에서는 MCU PC10, FPGA 확장 RX의 XDC, 공통 GND, 9600 8N1을 순서대로 점검할 필요가 있다.[1]

넷째, FPGA 시험은 실행 시간을 줄인 매개변수를 사용하였다. 서보의 μs 파형을 하드웨어 ms 측정으로 해석하지 않고, DAC 갱신 시간도 TB와 보드 조건을 나누어 계산하였다.

7.3 결론

과제5·6의 문자 대조로 통신 경로를 확인하였고, 과제7과 서보·SRAM·DAC 영상에서 실제 장치의 표시 또는 움직임을 관찰하였다. 디지털 제어 결과와 보드 관찰은 서로 보완적이다. 같은 조건에서 재현하려면 실제 배선과 FPGA 핀 제약을 함께 확인해야 한다.

7.4 전체 소스와 시연 링크

소스 폴더 / FPGA 파형

사진·보드 영상

보드 영상	길이
servo_video.mp4	8.07 s
sram_video.mp4	33.90 s
dac_video.mp4	48.00 s
과제7_video.mp4	8.63 s

영상 캡처의 시간은 각 원본 영상의 시작점을 0 s로 한 값이다. 세로 영상 사진은 보기 위해 90도 회전하였다. 원본 사진 12개와 네 개의 영상은 위 경로에서 확인할 수 있다. MCU의 세 과제 모드는 Task5·Task6·Task7로 구분한다.

7.5 제출 코드 확인

MCU 소스의 각 모드는 Arm GNU Toolchain으로 컴파일·링크하여 확인하였다. 이 빌드는 새 펌웨어의 보드 다운로드 시험이나 촬영 당시 바이너리의 동일성 검사를 대신하지 않는다. FPGA 결과 분석은 첨부된 Vivado 파형·로그와 RTL 검토를 근거로 하였다.

참고문헌

[1] STM32 실습 매뉴얼, 「STM32_2 · 컴퓨터와 FPGA 통신」, 10/26. 주차 교안. 과제5·6·7, 통신 연결 및 FPGA 개별 실습 목표.

[2] 「FPGA 추가 실습 · 모듈과 테스트벤치」, 서보·SRAM·DAC. 모듈 설명. 시간 축소 시험과 외부 장치 제어.

[3] Pong P. Chu, FPGA Prototyping by Verilog Examples: Xilinx Spartan-3 Version, Wiley, 2008. ISBN 9780470185322. 제8장 UART 및 제11장 External SRAM. DOI: 10.1002/9780470374283.

[1]과 [2]는 실험 절차·설계 조건의 근거이며, [3]은 UART 및 외부 SRAM 회로 구성에 관한 ISBN 도서 참고자료이다. 사진과 영상에 대한 관찰은 해당 그림에 제시하였다.

부록. 시뮬레이션 확대·전체 파형

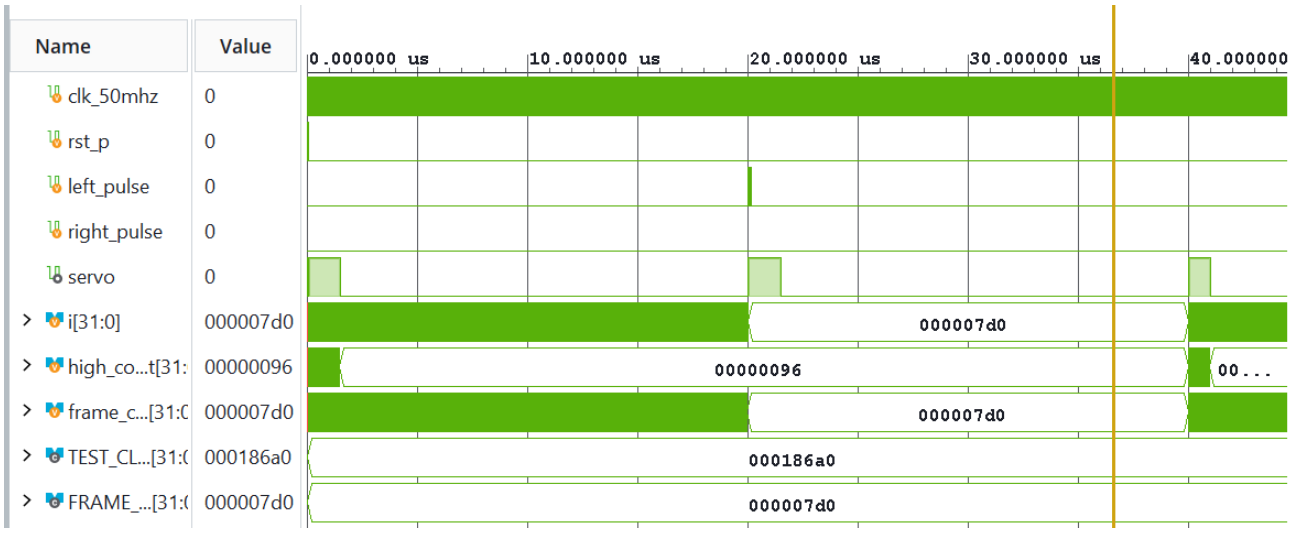


그림 A1. 서보 초기 150클록 HIGH 구간과 좌측 입력에 따른 폭 변화의 상세 화면.

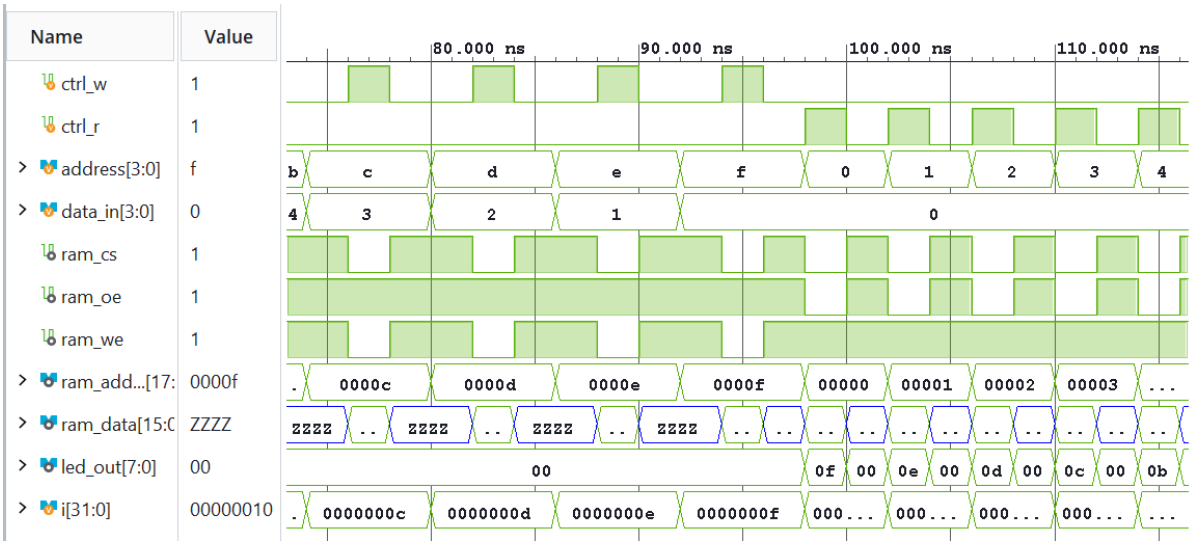


그림 A2. SRAM 읽기 확대. 주소별 LED 값과 접근 사이의 고임피던스 구간.

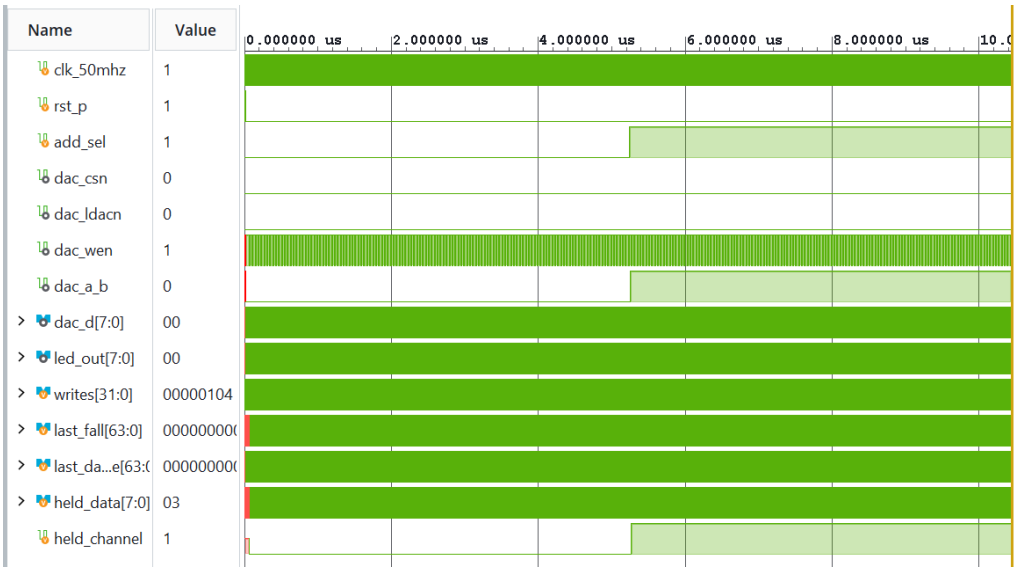


그림 A3. DAC 전체 구간. 채널 선택 변화 및 반복적인 쓰기 신호.

부록의 파형은 본문 그림 4·7·10의 시간 범위와 확대 정도를 보완한다. 신호명·진법·시간 단위를 함께 읽어 디지털 시험 조건을 해석하였다.